FRED TALKS

9th November 2026

CONTENTS

The lies I used to tell myself
CATE HALL · 1900 WORDS

A statistical approach to model evaluations
ANTHROPIC.COM · 1598 WORDS

Ownership
THORSTEN BALL · 1122 WORDS

Writing as a thinking tool
NOREPLY@BLOGGER.COM (MURAT) · 1670 WORDS

The lies I used to tell myself

CATE HALL · 23 JAN 2026 · SOURCE



1. What doesn’t kill you makes you stronger.

When I was 31, my husband of four years, who I loved more than my own life, left me completely out of the blue. We’d had an incredible, epic romance — drawn together like magnets, we married on our third date and spent the years that followed burrowing ever closer together. Maybe that was the problem — I can’t say for sure, because I never got a satisfying explanation out of him. One day I was the protagonist of a Princess Bride-worthy love story; 48 hours later, he moved out.

From the time we met, I had never contemplated the possibility of living without him, and I didn’t want to. I wanted to die for a long time — at least a couple of years. But as I slowly learned to sequester away thoughts of him and the emotions they brought up, I became convinced of a silver lining: The experience would make me impenetrable. If I survived it, I would know I could survive anything, and nothing would be able to hurt me again.

It took the better part of a decade and another deeply identity-shifting relationship, with the man who became my second husband, for me to see that I had it all wrong: Impenetrability means avoiding the parts of life that are most life-like. There’s a reason the visual metaphor for love is an arrow.

The near-fatal catastrophes of our lives don’t always make us stronger. Surviving one hard thing does provide evidence that you can survive other hard things, which is easy to *mistake* for getting stronger. But in reality they often just make us brittle, make us dissociate and flee from the situations that remind us we can be broken.

Rule 3: Keep it simple, plain, and succinct

I am an Elements of Style guy. Strunk and White got it right, and their writing still reads fresh decades later. I adore Vonnegut too. His writing is simple as fuck and completely eloquent. It took a lot of balls to write that plainly. Complexity in prose is usually a place to hide sloppy thinking, or a way to weasel out of taking a stance. If you can't say it plainly, you probably haven't thought it through yet. And the way you think it through is by writing/editing.

Rule 4: Write without expectations

Write for yourself. The audience I have in mind is me. If others read it, that is a bonus, but the motivation has to be intrinsic, because the external rewards are far too delayed and too random to steer on. Writing like no one is watching also frees you up, and you can decide afterwards how much to share. Get your dopamine hit from the writing itself, not from people reading or sharing what you wrote, as that is very unreliable.

I have been writing and sharing on this blog since 2011, and around 2016 I started running into people at conferences who read it. I assumed it was a fluke, but it kept happening. Writing with a human voice turns out to get a lot of attention, and I am still surprised by how much. Paradoxically, you seem to get an audience only if you weren't doing it for the audience.

Rule 5: Put it on the schedule

You should stop trying to get inspiration to write and start scheduling it. Waiting for inspiration doesn't work, instead, putting your butt on seat and writing breeds good ideas for writing.

I find that keeping a posting schedule for the blog keeps me in "a writing frame of mind", which is really "a noticing frame of mind". Many weeks I tell myself a topic isn't interesting, but I start writing about it anyway, and then, what do you know, I am startled by the fount of interesting things that surfaces.

I also feel relieved after writing something as the thought is out of my system, I have closure on it, and I have room to think new thoughts. If a week goes by without a post, I feel clogged. So put writing on the calendar and keep it there until it no longer needs to be on the calendar.

2. When you know, you know.

Mario Gabriele of the Generalist had a great “50 Things” style post a few months ago that I found myself nodding along to almost the whole way through. The glaring exception was Number 33: “Love is easy. The difference between bad relationships and meeting your spouse is not a matter of degree, but category. Compatibility is unignorable and effortless.”

I believed this completely, once. I’ve since come to believe that what’s unignorable and effortless is chemistry. Chemistry can be powerful enough to turn your life sideways, but it’s not the same thing as compatibility. Compatibility is litigated on the timescale of years and decades; it’s not a momentary feeling but a description of what happens during an extended course of change. Do you grow toward one another, or do you grow apart? Does the relationship help you move closer to the person you aspire to be?

I “just knew” with my first husband. I wasn’t sure with my second. Four years in, the signs have flipped.

3. If it were a good idea, someone would be doing it already.

There’s a belief, especially common among wonky types, that you don’t find $20 bills on the sidewalk — if an opportunity were real, someone would have seized it already. This sounds worldly and wise, but it’s actually a form of defensive thinking. It assumes the current way is best, that everything worth doing is being done.

In my experience, this is just false. When I started playing poker, I discovered that physical tells were basically a wide-open field — despite the fact that most pros believed the topic was played out. If you think you see an opportunity others have missed, you should have a story for why they’re missing it, and you should ask around to stress-test that story. But if no one can give you a satisfying reason, don’t assume one exists. Sometimes the answer really is just inertia or fear.

4. If it’s worth doing, it’s worth doing well.

Almost nothing is worth doing “well.” Many things can and should be done to a minimal standard of quality, because anything more is a waste of effort that can be better allocated. The exceptions aren’t things that should be done “well,” but things that should be done to an exceptionally high standard, probably a much higher one than you see modeled by the people around you.

This is not semantics; most people really “do everything the way they do anything” — whether that’s in a low-effort or high-effort way. But the point of phoning it in on a lot of stuff is to save your energy to expend on the 1 in 1000 things that really matter … and then to actually spend it.

A recent example: I decided that I want to narrate my own audiobook for *You Can Just Do Things*. When I told a friend in the industry this and asked whether they knew any good coaches to work with, they said that hiring a coach wasn't normal or necessary, because my publisher would decide based on a sample whether I was good enough at reading to do it, and if so give me a producer to work with day-of. But it "1 in 1000" matters to me whether the audiobook is so good no one would ever think of returning it on account of the narration, so I'm not interested in the normal-shaped solution that constitutes doing it "well," I'm interested in the one that causes me to come to mind when someone asks this question. In this case, that means hiring multiple coaches and practicing for months.

5. If you like your job, you're doing something wrong.

Back when I was more alienated from my emotions, I thought about work like this: You have a budget of energy to spend, and the goal is to efficiently convert that energy into impact. You identify the activity that generates the highest impact per unit of energy, and then you spam that button for as many waking hours as you can stomach. Is it any wonder I was always burning out?

What I failed to understand is that energy is not fixed. It's dramatically affected by what you choose to do — some activities increase your energy over time, others deplete it. This feels so obvious to me now that I feel kind of stupid including it here, but I know many people (they are endemic to the Bay Area) who still labor under the false belief. Whether you enjoy what you're doing affects your ability to do it year in and year out, and dismissing this as "selfish" doesn't make it untrue. As my husband puts it, the best use of effortful motivation is to engineer your life to require less of it.

6. "All people are insane. They will do anything at any time, and God help anybody who looks for reasons."

This line, from Kurt Vonnegut, used to neatly encapsulate the way I understood other people — which is to say, not at all. Trying to figure out why people did what they did, or imagine what they'd do next, seemed like a fool's errand. I was judgmental about it, too: I assumed this was a problem, not with me, but with everyone else.

I didn't fundamentally move beyond this belief until recent years, when I came into contact with the Enneagram. I imagine there are other personality typing systems that can do something similar, but the Enneagram was magical for me because it clued me into the fact that people can have very different motivational systems from my own that are still internally coherent and produce good things in the world.

Rule 1: Mess up

The first rule of writing club is to produce in bulk with no restraints at all. Don't criticize, don't fix the typos, don't write in full sentences if you don't feel like it. The goal in this phase is to just accumulate text and get to a zeroth draft you can work with.

McPhee argues best for why getting anything down matters so much. Even if you had a bad writing session, it gave your mind something to work with. After you get something down, you can spot problems with it and think of better ways of phrasing things. But none of that happens without the drafted version to react against.

This is also how research papers get written. Open a notes.org file and treat it as a lab notebook: write while you do the work, explore every tangent, produce without any pressure to be presentable. In the Köroğlu post I compared this to the foal fattened in a pitch-black stable before any training begins. The key part of that frame is the stable. During this phase you guard the idea from critics, including and especially yourself. Ideas start out fragile and easily squished. So suppress judgment for a while and see what the thing becomes.

Rule 2: Tidy up

Now you must become the critic, the editor. Forget that you wrote this. Imagine some other guy wrote it, and do a harsh review. Cut ruthlessly. Take delight in deleting text the way you would in deleting code. If you cannot bear to delete a good sentence, move it to a junk buffer at the bottom of the file so you can tell yourself you might bring it back. Spoiler: you won't.

Don't be discouraged by how bad the first draft was. McPhee says the way to do a piece of writing is three or four times over, never once. This is good news. Thinking in multiple drafts takes the pressure off: you don't have to be spontaneous, brilliant on the first pass, or finished today.

I think the reason writing works so well for thinking is that it slows you down while keeping you on the problem. In that extra time, the insights and connections get room to surface. Chunking/abstracting happens in the background. Editing is then the magic trick where you pretend you had an organized thought all along. It is the packaging of the product.

Sometimes the unearthing goes further than understanding one paper. In 2018, I wrote a quick post about the anatomical similarities between Paxos and Nakamoto consensus. I had not started with that thesis originally. I started writing, and the connections got clearer as I went, and that post became the seed for one of our research papers. This is not uncommon, this is called freewriting. This is what happens when you give the machine a tape.

Writing is also the reason I remember anything. I refer back to my own blog constantly. Because I strained my brain writing those posts, rereading them re-fires the same neurons, and I reconstruct the understanding quickly. In this sense, the blog became my external memory. The thoughts I never wrote down do not stick well, because they were never fully thought.

> If you think without writing, you only think you're thinking.
>
> -- Leslie Lamport

Writing in the age of LLMs

Is writing for thinking getting obsolete now since you can now argue anything with a model that has read everything?

Well, the model has read everything except what is in your brain! It knows the literature, the standard framing, and the consensus. But it does not know your half-formed intuition about why the standard framing may be wrong for this particular case. AI made a lot of progress, but I don't trust an LLM to take the creative leap and make a new connection. I think that would require another AI breakthrough. So, if you open the chat immediately on anything, you lose the opportunity to write/think for yourself.

This is a real problem, and it seems to be spreading quickly. We have been celebrating Amazon's six-pager as a great example of writing-for-thinking. Bezos had said: "There is no way to write a six-page, narratively structured memo and not have clear thinking." Unfortunately, with LLMs, that sentence has stopped being true. You can now produce six pages with no thinking at all, and the six pages still look like evidence that the thinking happened.

A model optimizing next-token probability cannot do your thinking. If you trust it to hold the pen, no thinking would get done. As everyone turns their thinking down and becomes a meat-proxy for generated text, if you still keep writing yourself, you would still be thinking for yourself. So writing is now a comparative advantage, and a widening one!

Anyways, enough about why you should write. Here is my advice on how you should write.

People's behavior was previously incomprehensible to me because I was imputing my own motivational system to them, trying to figure out what would cause me to act the way they did — essentially, concluding that *their* actions made no sense in light of *my* goals. But people have wildly diverse emotional hardware, which determines what's rational for them. And my particular personality — independent, moralistic, systematic, challenge-seeking, competitive — is unusual, so I was especially poorly placed to measure others by the yardstick of myself.

This shift in perspective was dramatically good for me and for my relationships with other people. It let me see the ecosystem of psychologies as a kind of miracle — wow, it really does take all kinds — rather than a prompt for cosmic horror. And it gave me confidence that, if I invested in understanding people more deeply, they would eventually cease to be total mysteries, prone to bizarre and unreasonable behavior. As far as I can tell, this is the better part of emotional intelligence.

7. Money can't buy happiness.

The good version of this advice is: You'll still have to work at being happy, even if you're rich. Money in your bank account won't do it for you, and wealth comes with its own pathologies, like endless status-chasing. But the common gloss — that money stops mattering past some modest threshold — is basically false.

You may have heard there's research showing money doesn't buy happiness. That research was explosively popular precisely because it was counterintuitive — it betrayed the commonsense intuition that more money means more freedom, more options, more ability to help people you care about. Turns out it was also riddled with methodological problems. More recent work by Matthew Killingsworth (nominative determinism FTW), using large-scale studies that pinged participants about their happiness at random moments, found steady returns to well-being with rising income, no plateau in sight. The myth persists because it serves a psychological function: It helps people without money feel better about their situation, and helps the wealthy downplay their advantages. But it's not true.

8. Intuition is fake and rationality solves everything.

I used to be horribly judgmental of people who made decisions based on intuition, as in, "I just do what feels right." And then I noticed that whenever I overrode my intuitions, I made *terrible* decisions. I hired the wrong people, started projects with the wrong people, dated the wrong people, ate food that made me feel bad, did forms of exercise I found unfun and injurious. I don't have a satisfying theory for why this is, but I know what happens: When I shift from emotional intuition to a logical story about a decision, it's much easier for my reasoning to get hijacked by plausible-sounding directives that turn out to be nonsense.

It took me a long time to learn this, and that might have something to do with my training in law and poker. Overriding intuition in favor of explicit reasoning makes sense in contexts that reward systematic rationality, where you will get separated from your money if you count on gut feelings. But most of life isn't a closed system with enumerated rules and clear outcomes. If you try to explicitly name all the factors that make someone a good friend or co-founder, you might come up with some good indicators, but your crude map will also mislead you about the territory.

This is why "wisdom is knowledge that can't be transmitted." Wise people have navigational skill, not a long list of rules.

***Sign up to be notified** when my book, You Can Just Do Things, is available for purchase.*

A statistical approach to model evaluations

ANTHROPIC.COM · 14 SEP 2025 · SOURCE

Suppose an AI model outperforms another model on a benchmark of interest—testing its general knowledge, for example, or its ability to solve computer-coding questions. Is the difference in capabilities real, or could one model simply have gotten lucky in the choice of questions on the benchmark?

With the amount of public interest in AI model evaluations—informally called "evals"—this question remains surprisingly understudied among the AI research community. This month, we published a new research paper that attempts to answer the question rigorously. Drawing on statistical theory and the experiment design literature, the paper makes a number of recommendations to the AI research community for reporting eval results in a scientifically informative way. In this post, we briefly go over the reporting recommendations, and the logic behind them.

Recommendation #1: Use the Central Limit Theorem

Evals often consist of hundreds or thousands of unrelated questions. MMLU, for instance, contains questions as diverse as:

- Who discovered the first virus?

> You know what TURING MACHINES can do.
>
> For example, Finite Automata can add but not multiply.
>
> Turing Machines can compute any computable function.
>
> Turing machines are **incredibly more powerful** than Finite Automata.
>
> Yet the only difference between a FA and a TM is that
>
> the TM, unlike the FA, has paper and pencil.
>
> Think about it.
>
> It tells you something about the power of writing.
>
> Without writing, you are reduced to a finite automaton.
>
> With writing you have the extraordinary power of a Turing machine.

This is a crisp technical claim about which problems you can solve with writing. If you take the paper away, you downgrade yourself by an entire complexity class!

I think this explains a lot about human history as well. Language let us externalize some of our thinking, and our species got smarter. Writing let us externalize even more of it, and we got smarter again. I suspect this is also why LLMs are as capable as they are: so much of our intelligence is already deposited in language that a model trained on it inherits a good deal of intelligence for free.

Out-of-core execution

When we write, we are externalizing our thinking onto paper, and that is what raises our capability to think bigger thoughts. I liken this to out-of-core execution, or external memory algorithms.

A good example of this for me is reviewing a paper. When I read a paper, I think I understand it, and then I sit down to write the review and find out I understood maybe 60% of it. While writing the review, I end up contextualizing the paper better, and get to understand it fully. And sometimes I even realize the ideas are bigger than the authors noticed, and they connect to many other things.

In fact, I don't expect *anybody* to *always* do *all of these things* for *everything*. It's a mental checklist of things to consider — problem, edge cases, tradeoffs, deployment, customers, messaging, … — but for quite a few things there aren't edge cases to consider. Or big tradeoffs to weigh. Or deployment is a solved problem. And maybe someone else does the messaging for you.

And I imagine that most of these things you shouldn't even consider when you work in a company with, say, 5000 employees. I've never worked in a company that large, only startups, so I can't speak to how to successfully ship a software feature at Apple, end to end.

But when you work in a small company in which there's only a single department, when you want to build things you're proud of, when you work with me and you say own something, I expect you to keep these things in mind and run through them before you declare something as done.

You know what *you* should own? A subscription to this newsletter:

Writing as a thinking tool

NOREPLY@BLOGGER.COM (MURAT) · 07 NOV 2026 · SOURCE

In middle school I could not get through long passages no matter how hard I tried. I would read an entire page and find that nothing had registered, so I started taking notes out of desperation. This was the late 80s, and Tony Buzan's mind maps had just reached Turkey. This led me to produce pages and pages of notes that looked like spiderwebs. Creating those notes have helped me tremendously. I made sense of what I was reading by figuring out which branch they would hang off in my mind maps. I realized my pages were not for storing the information as much as they were for processing it.

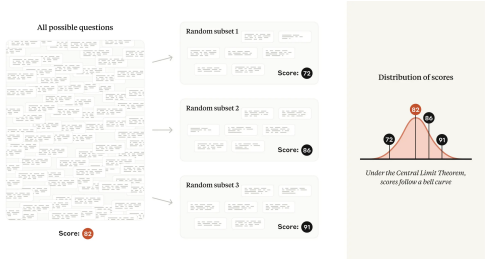
It took me another ten years to understand why this works so well. During a stretch of PhD procrastination, I read Manuel Blum's advice to graduate students:

> You are all computer scientists.
>
> You know what FINITE AUTOMATA can do.

• What is the inverse of $f(x)=4-5x$?

• Who said that "Jurisprudence is the eye of law"?

To compute an overall eval score, each question is separately scored, and then the overall score is (usually) a simple average of these question scores. Typically, researchers focus their attention on this observed average. But in our paper, we argue that the real object of interest should not be the *observed* average, but rather the *theoretical* average across all possible questions. So if we imagine that eval questions were drawn from an unseen "question universe," we can learn about the average score in that universe—that is, we can measure the underlying *skill*, independent of the "luck of the draw"—using statistical theory.



If we imagine that eval questions were drawn from a "question universe," then eval scores will tend to follow a normal distribution, centered around the average score of all possible questions.

This formulation buys us analytic robustness: if a new eval were to be created with questions having the same difficulty distribution as the original eval, we should generally expect our original conclusions to hold.

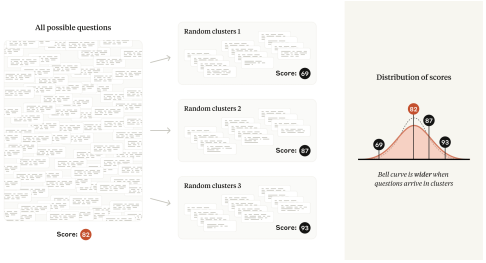
In technical terms: under the fairly mild conditions of the Central Limit Theorem, the mean values of several random samples taken from the same underlying distribution will tend to follow a normal distribution. The standard deviation (or width) of that normal distribution is commonly known as the standard error of the mean, or SEM. In our paper, we encourage researchers to report the SEM, derived from the Central Limit Theorem, alongside each calculated eval score—and we show researchers how to use the SEM to quantify the difference in theoretical means between two models. A 95% confidence interval can be calculated from the SEM by adding and subtracting 1.96 × SEM from the mean score.

Recommendation #2: Cluster standard errors

Many evals violate the above assumption of independently selected questions, and instead consist of groups of closely related questions. For example, several questions in a reading-comprehension eval may ask about the same passage of text. Popular evals that follow this pattern include DROP, QuAC, RACE, and SQuAD.

For these evals, each question's selection from the "question universe" is no longer independent. Because including several questions about the same passage of text will yield less information than selecting the same number of questions about different passages of text, a naive application of the Central Limit Theorem to the case of non-independent questions will lead us to underestimate the standard error—and potentially mislead analysts into drawing incorrect conclusions from the data.

Fortunately, the problem of clustered standard errors has been extensively studied in the social sciences. When the inclusion of questions is non-independent, we recommend clustering standard errors on the unit of randomization (for example, passage of text), and we provide applicable formulas in our paper.



If questions arrive in related clusters—a common pattern in reading-comprehension evals—eval scores will be more spread-out compared to the non-clustered case.

In practice, we have found that clustered standard errors on popular evals can be over three times as large as naive standard errors. Ignoring question clustering may lead researchers to inadvertently detect a difference in model capabilities when in fact none exists.

Recommendation #3: Reduce variance within questions

Variance is a measurement of how spread-out a random variable is. The variance of an eval score is the square of the standard error of the mean, discussed above; this quantity depends on the amount of variance in the score on each individual eval question.

- Test it manually. Yes, there's automated tests. But in 99% of cases you can manually test or confirm that what you built works: you can run it yourself, you can ask an agent to run through test scenarios, you can poke at the data before and after, you can take screenshots, you can make a demo. Are you sure that what you did actually solves the problem?

- Make sure it lands in production and *works in production*. Is it deployed? Did the deploy fail? Do you need to activate a feature flag? Does the feature flag work? Can you use it in production? Can you confirm it's actually deployed?

- If you think your colleagues needs to know about this change, because it's new feature they should all test, or it's a new convention in codebase, or maybe it's a tricky thing everybody needs to be aware of, or something else: let them know! Do not underestimate peripheral vision: knowing that person X yesterday changed the behavior of how Z works might save person Y three hours of debugging today when a bug report related to Z comes in.

- Do customers need to know? Who reported the bug? Who's blocked? Let them know.

- Does the world need to know? Announce that it's out.

- Are there follow-ups? Do you need to check on what you shipped in the logs? A week later maybe?

Yes, that's a lot. And there's actually more, because I'm sure I forgot some stuff.

But that's how you build a product in a small team. We don't have PMs, we don't have a Q&A department. We're small, but we're *great*, we can do all of that.

And it's always okay to ask for help, it's okay to ask questions, it's okay to redo things and triple-check. What's not okay is to implicitly assume that someone else will do the things here that you haven't thought about.



"How does this apply juniors? You can't expect them to really do all of that?"

I've been asked these questions, or variations of them, after I shared the thoughts above and here's my answer.

I do *not* expect juniors to *do* all of these things right away. But I would expect them to read through the list and *aspire* to one day be able to do all of these things. Until then, they can and should ask for help.

Just had a (great) conversation about ownership and engineering here and I realized that I often use the phrase “ownership” or allude to it, but haven’t explained what “ownership” means to me in a while.

So, ownership.

If I ask you “can you own this?” or “can you take care of this?” or “are you on it?” — what I’m doing is I’m asking you to *own* it, to own the solution of a problem from end to end. From “we have a problem” to “we don’t have to think about it again.”

That means, when you say that you’re owning something, the expectation is that you…

- Think about what the problem actually is. Maybe you already have a solution in mind, without having thought about what we’re actually trying to solve here. Maybe you think “the problem is that we need to migrate from using X to using Y”, but that’s not a problem, that’s a solution. The problem is likely something like “performance is bad”, “it’s not stable”, “it fails for customer x”. Maybe there’s other possible solutions to that? Think about those. What are the tradeoffs? What’s the best solution to go with considering these tradeoffs?

- Think about edge cases. What are they? Which ones are important? Which ones can we ignore?

- Think about failures. Network failures are a given, for example. How do we handle them? Retry? Well, how often? How long?

- Think about data flow. How much data is involved here? Does data need to be migrated? Cleaned up? How can I get my hands on data to properly test this? What invariants are in the data? What assumptions do I have about the shape of the data that I haven’t confirmed yet?

- Think about how you’d test this. How can I know that what I built is correct or not? Are tests enough? Do I need to manually poke at things? Is the difference visible on a screenshot or in a video?

- How would we announce this? How do we communicate it? Can you picture it? How does it fit into the larger picture of our roadmap? Questions or concerns in that area — push back! ask!

- Do the work, with precision, with care, with a sense of urgency, with calmness. Do not half-ass things. Before you merge, ask yourself: am I proud of this? would I show this to John Carmack and say “here’s what I built, under these constraints, with these tradeoffs?”

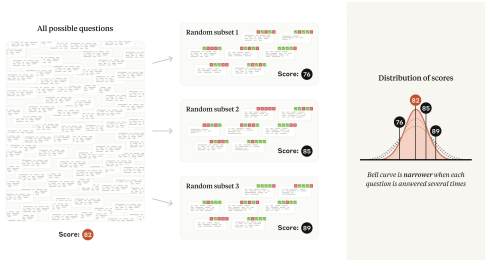
A key insight of our paper is to decompose a model’s score on a particular question into two terms that are added together:

- The mean score (the average score that the model would achieve if asked the same question an infinite number of times—even if the model might produce a different answer each time); and

- A random component (the difference between a realized question score and the mean score for that question).

Thanks to the law of total variance, reducing the variance in the random component directly leads to a smaller standard error of the overall mean, and thus greater statistical precision. Our paper highlights two strategies for reducing variance in the random component depending on whether or not the model is asked to think step by step before answering (a prompting technique known as CoT, or chain-of-thought reasoning).

If an eval uses chain-of-thought reasoning, we recommend resampling answers from the same model several times, and using the question-level averages as the question scores fed into the Central Limit Theorem. We note that the Inspect framework correctly computes standard errors in this way via its *epochs* parameter.



If a model produces answers non-deterministically, then generating (and grading) several answers per question will result in less spread-out eval scores.

If the eval does not use chain-of-thought reasoning (i.e., its answers are not “path dependent”), we note that the random component in the score may often be eliminated altogether using next-token probabilities from the language model. For example, if the correct answer to a multiple-choice question is “B”, we would simply use the probability of the model producing the token “B” as the question score. We are not aware of an open-source evals framework which implements this technique.

Recommendation #4: Analyze paired differences

Eval scores don't have any meaning on their own; they only make sense in relation to one another (one model outperforms another model, or ties another model, or outperforms a person). But could a measured difference between two models be due to the specific choice of questions in the eval, and randomness in the models' answers? We can find out with a two-sample *t*-test, using only the standard errors of the mean calculated from both eval scores.

However, a two-sample test ignores the hidden structure inside eval data. Since the question list is shared across models, conducting a paired-differences test lets us eliminate the variance in question difficulty and focus on the variance in responses. In our paper, we show how the result of a paired-differences test will be related to the Pearson correlation coefficient between two models' question scores. When the correlation coefficient is higher, the standard error of the mean difference will be smaller.

In practice, we find the correlation of question scores on popular evals between frontier models to be substantial—between 0.3 and 0.7 on a scale of −1 to +1. Put another way, frontier models have an overall tendency to get the same questions right and wrong. Paired-difference analysis thus represents a "free" variance reduction technique that is very well suited for AI model evals. Therefore, in the interest of extracting the clearest signal from the data, our paper recommends reporting pairwise information—mean differences, standard errors, confidence intervals, and correlations—whenever two or more models are being compared.

Recommendation #5: Use power analysis

The flip side of the statistical significance coin is statistical power, which is the ability of a statistical test to detect a difference between two models, assuming such a difference exists. If an eval doesn't have very many questions, confidence intervals associated with any statistical tests will tend to be wide. This means that models will need to have a large underlying difference in capabilities in order to register a statistically significant result—and that small differences will likely go undetected. Power analysis refers to the mathematical relationship between observation count, statistical power, the false positive rate, and the effect size of interest.

In our paper, we show how to apply concepts from power analysis to evals. Specifically, we show researchers how to formulate a hypothesis (such as *Model A outperforms Model B by 3 percentage points*) and calculate the number of questions that an eval should have in order to test this hypothesis against the null hypothesis (such as *Model A and Model B are tied*).

We believe that power analysis will prove helpful to researchers in a number of situations. Our power formula will inform evaluators of models about the number of times to re-sample answers from questions (see Recommendation #3 above), as well as the number of questions that may be included in a random subsample while retaining the desired power properties. Researchers might use the power formula to conclude that an eval with a limited number of available questions is not worth running on a particular pair of models. Developers of new evals may wish to use the formula to help decide how many questions to include.

Conclusion

Statistics is the science of measurement in the presence of noise. Evals present a number of practical challenges, and a true science of evals remains underdeveloped. Statistics can only form one aspect of a science of evals—but a critical one, as an empirical science is only as good as its measuring tools. We hope that the recommendations in our paper **Adding Error Bars to Evals: A Statistical Approach to Language Model Evaluations** will help AI researchers calculate, interpret, and communicate eval numbers with greater precision and clarity than before—and we encourage researchers in the AI community to explore other techniques from experiment design so that they may understand more exactly all the things that they want to measure.

Ownership

THORSTEN BALL · 08 JUL 2026 · SOURCE

Thoughts on ownership I sent to the Amp team in internal note

The following is an internal Slack message I sent to my Amp teammates after I had a conversation with one of them about ownership. It's only lightly edited.

I shared it before, but not here, because I didn't think too much of it. Then today, someone said their CTO shared my post with them and I thought: well, now I have to put it in the newsletter, don't I? So here we are.

Below, after the Slack post, I added some thoughts on juniors on how I see this advice applying to them.

